

Unit 2: CONCEPT LEARNING AND THE GENERAL-TO-SPECIFIC ORDERING

Part 2: Concept Learning as Search & FIND-S

Outline

What we cover

- **2.3** Concept Learning as Search
 - **2.3.1** General-to-Specific Ordering of Hypotheses
- **2.4** The FIND-S Algorithm
- **Summary** of 2.3 & 2.4

2.3 Concept Learning as Search

Concept Learning as Search

- **Learning as search:** find a hypothesis $h \in H$ that best fits the training examples.
- The **hypothesis representation** defines what the learner can ever represent (and therefore learn).
- Two key spaces:
 - **Instance space X :** all possible instances (attribute combinations).
 - **Hypothesis space H :** all candidate descriptions/rules the learner can output.

2.3 Concept Learning as Search

Training Examples for *EnjoySport* (Table 2.1)

Example	Sky	AirTemp	Humidity	Wind	Water	Forecast	EnjoySport
1	Sunny	Warm	Normal	Strong	Warm	Same	Yes
2	Sunny	Warm	High	Strong	Warm	Same	Yes
3	Rainy	Cold	High	Strong	Warm	Change	No
4	Sunny	Warm	High	Strong	Cool	Change	Yes

Table: Positive and negative training examples for the target concept *EnjoySport*.

2.3 Concept Learning as Search

EnjoySport: Size of the Instance Space $|X|$

Attributes: *Sky, AirTemp, Humidity, Wind, Water, Forecast*

Given:

- Sky has **3** possible values
- AirTemp, Humidity, Wind, Water, Forecast each have **2** values

$$|X| = 3 \times 2^5 = 3 \times 32 = 96$$

2.3 Concept Learning as Search

Hypothesis Space: Syntactically Distinct Hypotheses

Hypothesis form (6-tuple):

$$h = \langle \textit{Sky}, \textit{AirTemp}, \textit{Humidity}, \textit{Wind}, \textit{Water}, \textit{Forecast} \rangle$$

For each attribute constraint we allow:

- a **specific value** (e.g., Sunny)
- **?** meaning “any value”
- $\emptyset$ meaning “no value allowed” (matches nothing)

Counts:

- Sky: $3 + 1 + 1 = 5$
- Each binary attribute: $2 + 1 + 1 = 4$

$$|H|_{\text{syntactic}} = 5 \times 4^5 = 5 \times 1024 = 5120$$

2.3 Concept Learning as Search

Syntactic vs. Semantic Hypotheses

- Any hypothesis containing at least one $\emptyset$ represents the **empty concept** (classifies all instances as negative).
- So, many syntactically different hypotheses collapse to the **same meaning**.

Semantic counting:

- **1** empty concept (all hypotheses with $\emptyset$)
- Hypotheses with **no** $\emptyset$:
 - Sky: (3 values+?) = 4
 - Each binary attribute: (2 values+?) = 3

$$|H|_{\text{semantic}} = 1 + 4 \times 3^5 = 1 + 4 \times 243 = 973$$

2.3.1 General-to-Specific Ordering

Intuition with EnjoySport Hypotheses

Example hypotheses:

$$h_1 = \langle \text{Sunny}, ?, ?, \text{Strong}, ?, ? \rangle, \quad h_2 = \langle \text{Sunny}, ?, ?, ?, ?, ? \rangle$$

- h_2 imposes fewer constraints $\Rightarrow$ it matches **more** instances.
- Every instance satisfying h_1 also satisfies h_2 .
- Therefore, h_2 is **more general** than h_1 .

2.3.1 General-to-Specific Ordering

Formal Definition

Satisfaction: an instance $x \in X$ satisfies $h \in H$ iff $h(x) = 1$.

More-general-than-or-equal-to ($\succeq$):

$$h_j \succeq h_k \iff \forall x \in X, (h_k(x) = 1 \Rightarrow h_j(x) = 1)$$

Equivalently (as sets):

$$\{x : h_k(x) = 1\} \subseteq \{x : h_j(x) = 1\}$$

Strictly more general ($\succ$):

$$h_j \succ h_k \iff (h_j \succeq h_k) \wedge \neg(h_k \succeq h_j)$$

2.3.1 General-to-Specific Ordering

Partial Order and Incomparability

- The relation $\succeq$ forms a **partial order** over H :
 - Reflexive: $h \succeq h$
 - Antisymmetric: $(h_j \succeq h_k \wedge h_k \succeq h_j) \Rightarrow h_j \equiv h_k$
 - Transitive: $(h_a \succeq h_b \wedge h_b \succeq h_c) \Rightarrow h_a \succeq h_c$
- Not a total order: some hypotheses are **incomparable**.

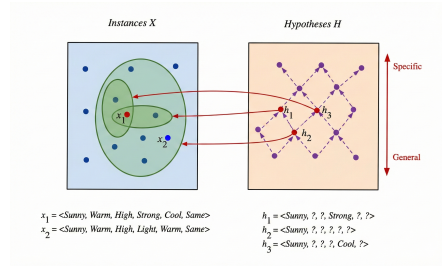


Fig. 2.1 / 2.2: Example general-to-specific structure

2.3.1 General-to-Specific Ordering

Real-World Examples: Total vs Partial Order

Total (Complete) Order: everything comparable

- Sorting people by **age**: for any two people A, B , either $age(A) \leq age(B)$ or $age(B) \leq age(A)$.

Partial Order: some pairs incomparable

- **Prerequisite structure** in a course/project:
 - Define $A \preceq B$ if task A must be completed before task B .
 - Two tasks can be independent (neither requires the other) $\Rightarrow$ **incomparable**.

Connection: hypothesis generality behaves like prerequisites—some hypotheses cannot be ranked as “more general” or “more specific” relative to each other.

2.3.1 General-to-Specific Ordering

Why This Ordering Helps Search

- The general-to-specific structure organizes H without enumerating every hypothesis.
- It supports efficient search via:
 - moving step-by-step (generalize / specialize)
 - pruning large regions of hypothesis space
 - reasoning about sets of consistent hypotheses
- Crucially, $\succeq$ depends only on which instances satisfy hypotheses, not on the target concept.

2.4 FIND-S: Finding a Maximally Specific Hypothesis

Key Idea

- Use the **more-general-than** partial order to organize search in H .
- Start with the **most specific** hypothesis in H .
- Whenever the hypothesis fails to **cover** a positive example, **generalize** it.

Cover (positive example): hypothesis h “covers” a positive example x if it correctly classifies it as positive (i.e., $h(x) = 1$).

2.4 FIND-S: Finding a Maximally Specific Hypothesis

Table 2.3: FIND-S Algorithm

FIND-S (pseudocode)

- ① Initialize h to the **most specific** hypothesis in H .
- ② For each **positive** training instance x :
 - ① For each attribute constraint a_i in h :
 - If the constraint a_i is satisfied by x , then do nothing.
 - Else replace a_i in h by the **next more general** constraint that is satisfied by x .
- ③ Output hypothesis h .

Note: FIND-S updates only on **positive** examples and ignores negative examples.

2.4 FIND-S

Trace on EnjoySport Training Examples (Step-by-Step)

Assume training examples arrive in the order (from Table 2.1):

- $x_1 = \langle \text{Sunny}, \text{Warm}, \text{Normal}, \text{Strong}, \text{Warm}, \text{Same} \rangle (+)$
- $x_2 = \langle \text{Sunny}, \text{Warm}, \text{High}, \text{Strong}, \text{Warm}, \text{Same} \rangle (+)$
- $x_3 = \langle \text{Rainy}, \text{Cold}, \text{High}, \text{Strong}, \text{Warm}, \text{Change} \rangle (-)$
- $x_4 = \langle \text{Sunny}, \text{Warm}, \text{High}, \text{Strong}, \text{Cool}, \text{Change} \rangle (+)$

Initialize to most specific:

$$h_0 = \langle \emptyset, \emptyset, \emptyset, \emptyset, \emptyset, \emptyset \rangle$$

2.4 FIND-S

Update on First Positive Example x_1

Since h_0 is too specific, generalize each $\emptyset$ to match x_1 :

$$h_0 \rightarrow h_1 = \langle \text{Sunny}, \text{Warm}, \text{Normal}, \text{Strong}, \text{Warm}, \text{Same} \rangle$$

- Now h_1 is **very specific**: it accepts essentially only the observed x_1 .

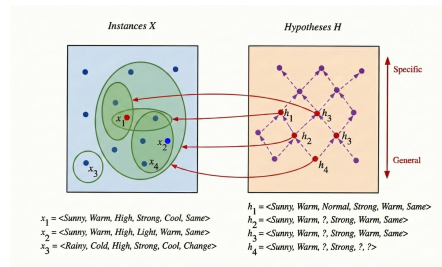


Fig. 2.2: Hypothesis space search in FIND-S

2.4 FIND-S

Update on Second Positive Example x_2

Compare x_2 with current h_1 :

$$x_2 = \langle \text{Sunny}, \text{Warm}, \text{High}, \text{Strong}, \text{Warm}, \text{Same} \rangle$$

Only Humidity differs (Normal vs High), so minimally generalize that attribute:

$$h_1 \rightarrow h_2 = \langle \text{Sunny}, \text{Warm}, ?, \text{Strong}, \text{Warm}, \text{Same} \rangle$$

- Rule: if an attribute value in h is not satisfied by the new positive example, replace it by ?.

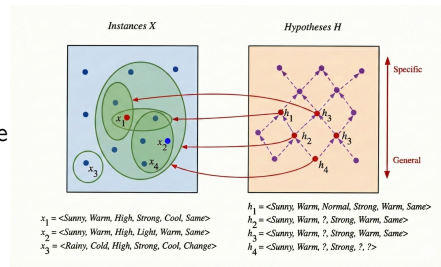


Fig. 2.2: Hypothesis space search in FIND-S

2.4 FIND-S

Encountering a Negative Example x_3 (No Change)

Third example is negative:

$$x_3 = \langle \text{Rainy}, \text{Cold}, \text{High}, \text{Strong}, \text{Warm}, \text{Change} \rangle \quad (-)$$

FIND-S makes **no update**:

$$h_3 = h_2 = \langle \text{Sunny}, \text{Warm}, ?, \text{Strong}, \text{Warm}, \text{Same} \rangle$$

- FIND-S **ignores every negative example**.
- Here h already classifies x_3 as negative, so no revision is needed.

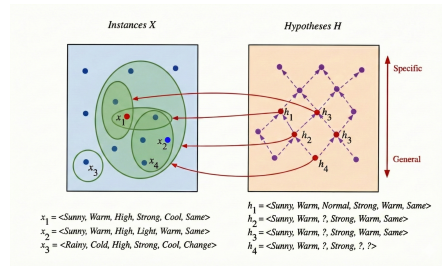


Fig. 2.2: Hypothesis space search in FIND-S

2.4 FIND-S

Why Ignoring Negatives Can Be Safe (Under Assumptions)

Assume:

- The true target concept c is in H
- Training data has **no noise/errors**

Then:

- h is the **most specific** hypothesis consistent with observed positives.
- Since c is also consistent with those positives, we must have:

$$c \succeq h$$

- c never covers a negative example $\Rightarrow h$ will also never cover that negative example (by definition of **more-general-than**).

Therefore: no revision is required for negative examples (when assumptions hold).

2.4 FIND-S

Update on Fourth Positive Example x_4

Fourth example is positive:

$$x_4 = \langle \textit{Sunny}, \textit{Warm}, \textit{High}, \textit{Strong}, \textit{Cool}, \textit{Change} \rangle (+)$$

Generalize only where needed:

$$h_2 = \langle \textit{Sunny}, \textit{Warm}, ?, \textit{Strong}, \textit{Warm}, \textit{Same} \rangle \rightarrow h_4 = \langle \textit{Sunny}, \textit{Warm}, ?, \textit{Strong}, ?, ? \rangle$$

- Water: Warm $\rightarrow$?
- Forecast: Same $\rightarrow$?

2.4 FIND-S

How FIND-S Uses the Partial Order (Search View)

- FIND-S performs a search that moves:

Most Specific → **Progressively More General**

- It follows **one chain** in the partial order (no branching/backtracking).
- At each step, it generalizes **only as far as necessary** to cover the new positive example.
- Hence at every stage, h is the **most specific** hypothesis consistent with all positives seen so far.

2.4 FIND-S

Key Property (Guarantee) and Assumptions

Key property (for conjunctions of attribute constraints, like EnjoySport):

- FIND-S outputs the **most specific** hypothesis in H that is consistent with the **positive** examples.

Consistency with negatives holds if:

- the correct target concept is contained in H , and
- training examples are correct (noise-free).

2.4 FIND-S

Unanswered Questions / Limitations

- **Has the learner converged?**
 - FIND-S cannot tell whether it found the *only* consistent hypothesis or one among many.
- **Why prefer the most specific hypothesis?**
 - Could prefer most general, or intermediate generality—FIND-S does not justify the choice.
- **Are training examples consistent (noise)?**
 - Errors/noise can mislead FIND-S, especially because it ignores negatives.
- **Multiple maximally specific hypotheses?**
 - Some hypothesis spaces require **backtracking** (FIND-S follows only one chain).

Summary

2.3 Concept Learning as Search & 2.4 FIND-S

2.3 Concept Learning as Search

- Learning can be viewed as searching a hypothesis space H to find a hypothesis that fits the data.
- The hypothesis representation determines what the learner can express and therefore learn.
- Example (EnjoySport):
 - Instance space size: $|X| = 3 \times 2^5 = 96$
 - Syntactically distinct hypotheses: $|H|_{\text{syntactic}} = 5 \times 4^5 = 5120$
 - Semantically distinct hypotheses: $|H|_{\text{semantic}} = 973$

2.4 FIND-S: Finding a Maximally Specific Hypothesis

- Uses the general-to-specific partial order to move from the **most specific** hypothesis to more general ones.
- Updates only on **positive** examples by minimally generalizing h to cover them.
- Guarantee (for conjunctions of attribute constraints): outputs the **most specific** hypothesis consistent with all observed positives.

• Limitations: ignores negatives, cannot tell if it has converged to the true concept, sensitive to

Thank You!

Questions?